Dask and Ray
Python-First Distributed Data

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — Two Tools, Different Jobs

Session 1 gave you the theory. Now: the two Python-native tools that actually implement it, and when to reach for each.

THINK OF IT LIKE THIS: Dask is like a pandas notebook that learned to delegate — same recipes, more kitchens. Ray is more like a general contractor — hire workers (tasks) for any job, or keep a specialist (actor) on staff who remembers things between jobs, like a loaded model on a GPU.

1 Dask: scale your DataFrame code

Nearly identical to pandas. Best for tabular ETL at 10GB-10TB scale.

2 Ray: scale arbitrary Python

General-purpose tasks and actors. Best for GPU batch inference and ML pipelines.

Key Terms Before We Start

A quick shared vocabulary — skip ahead if these are all second nature to you.

Dask DataFrame — A pandas-like DataFrame split across a cluster; same API, runs in parallel.

Cluster / scheduler — The set of machines doing the work, and the process assigning tasks to them.

Ray task — A stateless function that Ray can run on any available worker.

Ray actor — A stateful Python object that stays alive on one worker across many calls.

Compute() / persist() — Trigger execution now, or cache an intermediate result in memory.

Dask — Three High-Level Collections

DataFrame

Pandas DataFrames split by row.
Same API: df.groupby, df.merge.
Best for tabular ETL.

Array

A chunked numpy ndarray. Same API: arr.mean, arr+arr.
Best for scientific computing.

Bag

Parallel iterable over Python objects. map, filter, reduce.
Best for raw JSON/text.

KEY IDEA: Dask's pitch is to scale your EXISTING pandas/numpy code with minimal changes — swap 'import pandas as pd' for 'import dask.dataframe as dd'.

Dask DataFrame — It Looks Like pandas

```
from dask.distributed import Client; import dask.dataframe as dd
client = Client('dask-scheduler:8786')                          # connect to any
cluster type

df = dd.read_parquet('s3://logs/events/')      # 1 TB across 500 files — LAZY
df = df[df.event == 'click']; df['hour'] = df.ts.dt.hour
hourly = df.groupby(['user_id','hour']).agg({'duration':'sum'})

hourly = hourly.persist()   # cache expensive intermediate across workers
out = hourly.compute()      # trigger execution, pull result to driver
```

- One line connects to any cluster type — local, Kubernetes, cloud-managed
- read_parquet returns a PLAN, not data — familiar API, ported transparently

Dask Deployment Models

LocalCluster	`Client()`	Dev — all cores of one machine.
Kubernetes	`KubeCluster(...)`	Dynamic — autoscaling pods.
Cloud managed	`Coiled / Saturn`	Managed Dask-as-a-service.

THINK OF IT LIKE THIS: Deployment model is separate from your code. Your ETL script stays identical — you just pass a different Client, like plugging the same appliance into a different outlet.

RECOMMENDATION: start with LocalCluster for dev, move to Kubernetes for production.

Ray — General-Purpose Distributed Python

```
# TASKS (stateless)
@ray.remote
def score(batch): return
model.predict(batch)
futures = [score.remote(b) for b in
batches]
results = ray.get(futures)
```

```
# ACTORS (stateful)
@ray.remote(num_gpus=1)
class InferenceWorker:
    def __init__(self): self.m =
load_model().cuda()
    def score(self, b): return self.m(b)
```

Ray Data Distributed, streaming, GPU-aware DataFrame.

Ray Train Distributed training wrapper (DDP/FSDP).

Ray Tune Hyperparameter search at scale.

Ray Serve Model serving with autoscaling, A/B.

Dask vs Ray — When to Pick Which

Scenario	Pick	Why
Tabular ETL at 100GB-10TB	Dask	DataFrame API matches pandas
Batch GPU inference on 1B images	Ray Data	GPU actors + streaming
Distributed hyperparameter search	Ray Tune	Purpose-built schedulers
Unified ML stack (train+tune+serve)	Ray	Tightly integrated libraries

Common in practice: use both. Dask for ETL -> persist to Parquet -> Ray for GPU inference.

Moving From pandas to Dask — What Doesn't Translate

The pitch ('just change pd to dd') is real for 90% of code — a few habits become traps at scale.

df.iterrows()

Defeats the whole point — pulls data row-by-row through Python. Use map_partitions instead.

df.sort_values() globally

A global sort needs a full shuffle to establish total order — avoid unless truly needed.

Missing pandas ops

Styler and some datetime edge cases aren't implemented — check the docs.

len(df) / df.shape

Both force a full pass over the data, just like .compute() would.

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Choosing Dask for GPU inference	Awkward workarounds, poor GPU utilisation	Use Ray Data/actors for GPU-heavy batch work
Using Ray tasks for a loaded model	Model reloads on every single call — very slow	Use a Ray actor to keep the model resident
Assuming Dask handles skewed keys well	One partition much larger than the rest	Same skew-handling as Spark — see Session 3

Quick Recap — Terms and Concepts

Dask — Scales pandas/numpy code with minimal changes — best for tabular ETL

Ray — General-purpose distributed Python — tasks (stateless) + actors (stateful)

Ray actor — Keeps state (like a loaded GPU model) resident across many calls

Split pipeline pattern — Dask writes Parquet, Ray reads it — object storage is the contract